

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
“Сибирский государственный университет телекоммуникаций и
информатики”

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

ОТЧЁТ
по дисциплине
“Веб-технологии”

по теме:
ПОГРУЖЕНИЕ В РАЗРАБОТКУ НА JAVASCRIPT

Студент:
Группа ИА-232

Гурачевский Никита

Преподаватель:
ст. преподаватель каф. ТС и ВС

А.В. Андреев

Новосибирск 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 ВЫПОЛНЕНИЕ ЛАБОРАТОРНОЙ РАБОТЫ	4
1.1 Цель работы	4
1.2 Выполненные задания	4
1.3 Публичная ссылка на проект	4
1.4 Результат выполнения	4
1.5 Листинг ключевых файлов	6
1.5.1 Серверная часть проекта	6
1.5.2 Главная страница	6
1.5.3 Клиентская логика JavaScript	7
1.5.4 Стили интерфейса	14
1.6 Выводы	18
ЗАКЛЮЧЕНИЕ	19

ВВЕДЕНИЕ

В лабораторной работе №13 рассматривается практическое применение JavaScript на нескольких уровнях: от базовых операций с переменными, строками и массивами до построения клиентского интерфейса и получения данных с внешнего API.

В качестве результата был подготовлен единый Node.js-проект, объединяющий решения по всем трём программным частям задания и отдельный блок с подготовкой к публикации приложения на хостинге.

1 ВЫПОЛНЕНИЕ ЛАБОРАТОРНОЙ РАБОТЫ

1.1 Цель работы

Цель работы — закрепить базовые и практические навыки разработки на JavaScript, включая работу с переменными, строками, массивами, DOM, API и подготовку проекта к публикации.

1.2 Постановка задания

Согласно PDF-заданию лабораторная работа №13 включает четыре части:

- реализацию задач по теме “Погружение в разработку на JavaScript. Базовые понятия”;
- выполнение заданий по теме “Основы JavaScript”;
- разработку страницы с получением данных с API и динамическим выводом карточек персонажей;
- подготовку проекта к публикации на хостинге и добавление результатов в отчёт.

1.3 Ход выполнения

1.3.1 Часть 1. Бонусная система интернет-магазина

В первой части реализован расчёт бонусного баланса пользователя. В коде созданы переменные ‘username’ и ‘bonusBalance’, выведена информация о пользователе, затем смоделировано изменение бонусов в течение 7 дней с учётом начисления 50 баллов за покупку раз в два дня и ежедневного списания 3 баллов. Результат выводится в отдельный блок интерфейса.

1.3.2 Часть 2. Работа с массивом сообщений

Во второй части создан массив сообщений чата, организован вывод переписки в хронологическом порядке с чередованием подписей “Друг” и

“Вы”. Дополнительно реализован поиск по тексту сообщений с помощью метода ‘includes’, после чего подходящие сообщения выводятся отдельным списком.

1.3.3 Часть 3. Разработка сайта с использованием API

В третьей части подготовлено веб-приложение, которое получает данные о персонажах через ‘fetch’ из внешнего API. После получения ответа формируются карточки персонажей и модальные окна с расширенной информацией. На случай недоступности API предусмотрен демо-режим с локальными тестовыми данными, что позволяет сохранить работоспособность интерфейса при отсутствии ответа сервера.

1.3.4 Часть 4. Подготовка к публикации

Для подготовки проекта к публикации приложение оформлено как Node.js-проект с Express-сервером. Статические ресурсы вынесены в каталог ‘public’, настроен запуск через ‘npm start’, а также добавлен служебный маршрут ‘/health’ для проверки работоспособности. В интерфейсе создан отдельный блок, описывающий готовность проекта к развёртыванию на Replit или другом Node.js-хостинге.

1.4 Результат выполнения

В результате выполнения лабораторной работы подготовлен единый проект, включающий все основные части задания. Главная страница содержит блоки с результатами по урокам 1 и 2, карточки персонажей и модальные окна для урока 3, а также отдельный блок с кратким описанием готовности проекта к публикации.

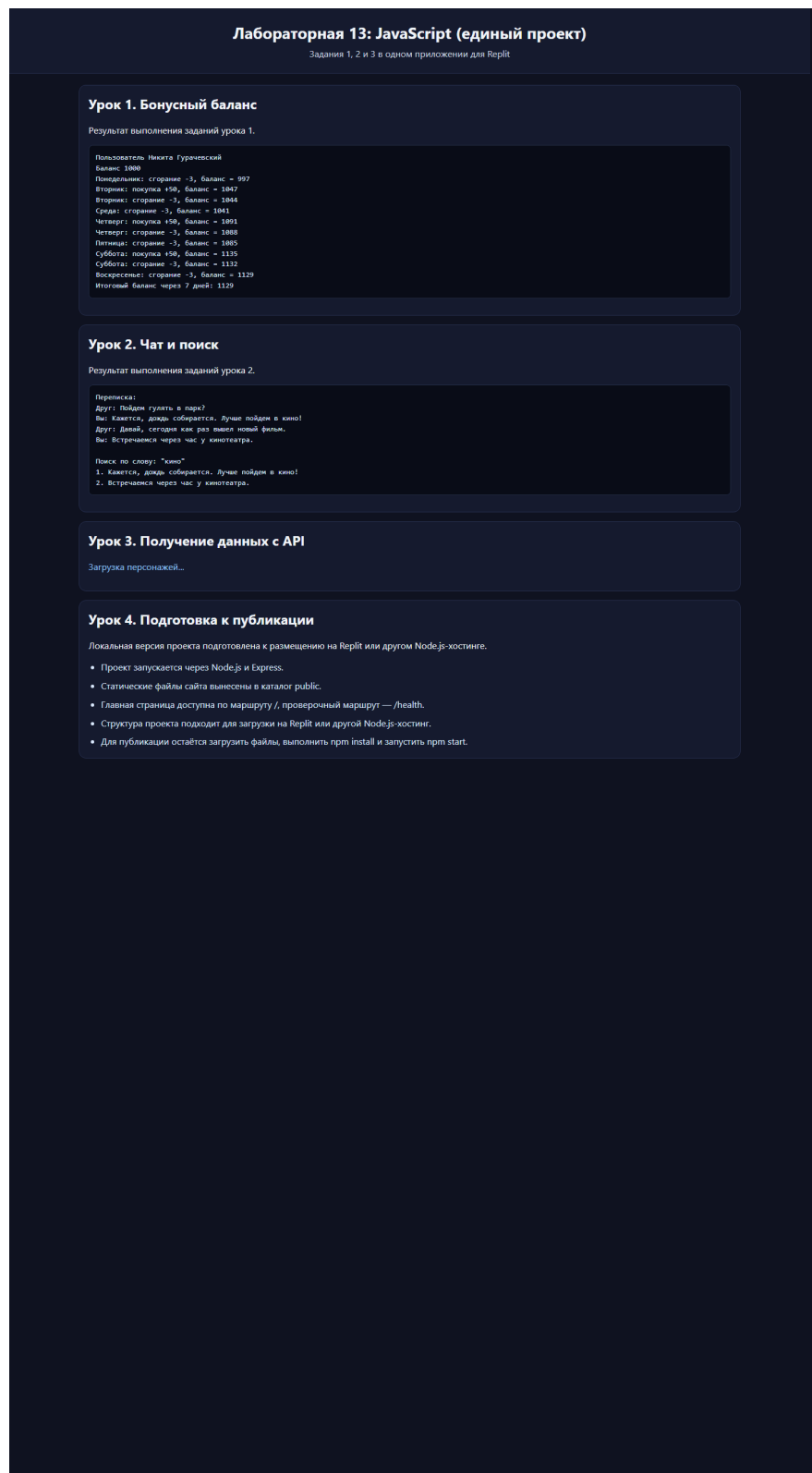


Рисунок 1 — Общий вид итогового проекта лабораторной работы №13.

1.5 Листинг ключевых файлов

1.5.1 Серверная часть проекта

```
1 const express = require("express");
2 const path = require("path");
3
4 const app = express();
5 const PORT = process.env.PORT || 3000;
6
7 app.use(express.static(path.join(__dirname, "public")));
8
9 app.get("/health", (req, res) => {
10   res.json({ ok: true });
11 });
12
13 app.listen(PORT, () => {
14   console.log(`Server started on port ${PORT}`);
15 });
```

Листинг 1.1 — Файл server.js.

1.5.2 Главная страница

```
1 <!doctype html>
2 <html lang="ru">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Лабораторная 13 - all in one</title>
7   <link rel="stylesheet" href="/css/styles.css">
8 </head>
9 <body>
10   <header class="header">
11     <h1>Лабораторная 13: JavaScript единый( проект)</h1>
12     <p>Задания 1, 2 и 3 водномприложениидля Replit</p>
13   </header>
14
15   <main class="container">
16     <section class="card">
17       <h2>Урок 1. Бонусныйбаланс </h2>
18       <p>Результат выполнениязаданийурока 1.</p>
```

```

19     <pre id="les1-output" class="console"></pre>
20 </section>
21
22 <section class="card">
23     <h2>Урок 2. Чатипоиск </h2>
24     <p>Результат выполнениязаданийурока 2.</p>
25     <pre id="les2-output" class="console"></pre>
26 </section>
27
28 <section class="card">
29     <h2>Урок 3. Получениеданныхс API</h2>
30     <p id="les3-status" class="status">Загрузка персонажей...</p>
31     <div id="les3-cards" class="cards"></div>
32 </section>
33
34 <section class="card">
35     <h2>Урок 4. Подготовкакакпубликации </h2>
36     <p>Локальная версияпроектаподготовленакразмещениюна Replit
        илидругом Node.jsхостинге-.</p>
37     <ul id="les4-output" class="deploy-list"></ul>
38 </section>
39 </main>
40
41 <div id="modals-root"></div>
42
43 <script src="/js/app.js"></script>
44 </body>
45 </html>

```

Листинг 1.2 — Файл public/index.html.

1.5.3 Клиентская логика JavaScript

```

1 const les1Output = document.getElementById("les1-output");
2 const les2Output = document.getElementById("les2-output");
3 const les4Output = document.getElementById("les4-output");
4
5 const runLes1Btn = document.getElementById("run-les1");
6 const runLes2Btn = document.getElementById("run-les2");
7
8 if (runLes1Btn) runLes1Btn.addEventListener("click", runLesson1);
9 if (runLes2Btn) runLes2Btn.addEventListener("click", runLesson2);
10

```

```

11 function runLesson1() {
12     const lines = [];
13
14     const username = "Никита Гурачевский";
15     let bonusBalance = 1000;
16
17     lines.push("Пользователь " + username);
18     lines.push("Баланс " + bonusBalance);
19
20     const bonusPerPurchase = 50;
21     const burnPerDay = 3;
22     const totalDays = 7;
23     const week = [
24         "Понедельник",
25         "Вторник",
26         "Среда",
27         "Четверг",
28         "Пятница",
29         "Суббота",
30         "Воскресенье"
31     ];
32
33     for (let dayIndex = 0; dayIndex < totalDays; dayIndex += 1) {
34         const dayName = week[dayIndex];
35         const hasPurchase = dayIndex % 2 === 1;
36
37         if (hasPurchase) {
38             bonusBalance += bonusPerPurchase;
39             lines.push(
40                 dayName + ": покупка +" + bonusPerPurchase + ", баланс = " +
41                     bonusBalance
42             );
43
44             bonusBalance -= burnPerDay;
45             lines.push(
46                 dayName + ": сгорание -" + burnPerDay + ", баланс = " + bonusBalance
47             );
48         }
49
50         lines.push("Итоговый баланс через " + totalDays + " дней: " + bonusBalance);
51         les1Output.textContent = lines.join("\n");
52     }
53 }

```

```

54 function runLesson2() {
55     const lines = [];
56
57     const messages = [
58         "Пойдем гулять в парк ?",
59         "Кажется, дождь собирается . Лучше пойдем в кино !",
60         "Давай, сегодня как раз вышел новый фильм .",
61         "Встречаемся через час у кинотеатра ."
62     ];
63
64     lines.push("Переписка:");
65     for (let i = 0; i < messages.length; i += 1) {
66         const author = i % 2 === 0 ? "Друг" : "Вы";
67         lines.push(author + ": " + messages[i]);
68     }
69
70     const searchText = "кино";
71     lines.push("");
72     lines.push('Поиск по слову : ' + searchText + '');
73
74     const foundMessages = [];
75     for (const message of messages) {
76         if (message.toLowerCase().includes(searchText.toLowerCase())) {
77             foundMessages.push(message);
78         }
79     }
80
81     if (foundMessages.length === 0) {
82         lines.push("Сообщения не найдены .");
83     } else {
84         for (let i = 0; i < foundMessages.length; i += 1) {
85             lines.push(i + 1 + ". " + foundMessages[i]);
86         }
87     }
88
89     les2Output.textContent = lines.join("\n");
90 }
91
92 function runLesson4() {
93     if (!les4Output) return;
94
95     const items = [
96         "Проект запускается через Node.js и Express.",
97         "Статические файлы сайта вынесены в каталог public.",

```

```

98     "Главная страница доступна по маршруту /, проверочный маршрут - /health.",
99     "Структура проекта подходит для загрузки на Replit или другой Node.js
    хостинг-.",
100     "Для публикации остаётся загрузить файлы , выполнить npm install
    и запустить npm start."
101 ];
102
103 les4Output.innerHTML = items.map((item) => "<li>" + escapeHtml(item) + "</li>
    ").join("");
104 }
105
106 const API_URLS = [
107     "https://jsfree-les-3-api.onrender.com/characters",
108     "https://jsfree-les-3api.onrender.com/characters"
109 ];
110
111 const FALLBACK_IMAGE = "https://via.placeholder.com/400x560?text=No+Image";
112 const statusEl = document.getElementById("les3-status");
113 const cardsEl = document.getElementById("les3-cards");
114 const modalsRoot = document.getElementById("modals-root");
115
116 function createAvatarDataUrl(name, bgColor) {
117     const initials = name
118         .split(" ")
119         .filter(Boolean)
120         .slice(0, 2)
121         .map((part) => part[0].toUpperCase())
122         .join("");
123
124     const svg =
125         '<svg xmlns="http://www.w3.org/2000/svg" width="600" height="840">' +
126         '<rect width="100%" height="100%" fill="' + bgColor + '"/>' +
127         '<text x="50%" y="50%" dominant-baseline="middle" text-anchor="middle" ' +
128         'font-family="Arial, sans-serif" font-size="180" fill="#ffffff">' +
129         initials +
130         "</text>" +
131         "</svg>";
132
133     return "data:image/svg+xml;utf8," + encodeURIComponent(svg);
134 }
135
136 const DEMO_CHARACTERS = [
137     {
138         id: "demo-1",

```

```

139     name: "Harry Potter",
140     species: "Human",
141     status: "Alive",
142     gender: "Male",
143     actor: "Daniel Radcliffe",
144     house: "Gryffindor",
145     image: createAvatarDataURL("Harry Potter", "#7c3aed"),
146     description: "Демоданные- дляофлайнрежима -."
147 },
148 {
149     id: "demo-2",
150     name: "Hermione Granger",
151     species: "Human",
152     status: "Alive",
153     gender: "Female",
154     actor: "Emma Watson",
155     house: "Gryffindor",
156     image: createAvatarDataURL("Hermione Granger", "#db2777"),
157     description: "Демоданные- дляофлайнрежима -."
158 },
159 {
160     id: "demo-3",
161     name: "Severus Snape",
162     species: "Human",
163     status: "Unknown",
164     gender: "Male",
165     actor: "Alan Rickman",
166     house: "Slytherin",
167     image: createAvatarDataURL("Severus Snape", "#0f766e"),
168     description: "Демоданные- дляофлайнрежима -."
169 }
170 ];
171
172 function normalizeCharacter(character, index) {
173     const imageFromThumbnail =
174         typeof character.thumbnail === "string"
175             ? character.thumbnail
176             : character.thumbnail &&
177                 character.thumbnail.path &&
178                 character.thumbnail.extension
179             ? character.thumbnail.path + "." + character.thumbnail.extension
180             : "";
181
182     const boolAlive =

```

```

183     typeof character.alive === "boolean"
184     ? character.alive
185     : typeof character.isAlive === "boolean"
186     ? character.isAlive
187     : null;
188
189     const statusFromAlive =
190     boolAlive === null ? "" : boolAlive ? "Alive" : "Deceased";
191
192     const speciesFromMarvel =
193     character.type ||
194     (character.comics && character.comics.length ? "Marvel Character" : "");
195
196     const statusFromMarvel = character.modified ? "Active" : "";
197
198     const universeFromMarvel =
199     character.universe ||
200     (character.series && character.series.length ? "Marvel Universe" : "") ||
201     "Marvel Universe";
202
203     return {
204     id: character.id || character._id || "character-" + index,
205     name: character.name || character.fullName || "Без имени",
206     type: character.type || character.species || character.race ||
207     speciesFromMarvel || "Не раскрыто",
208     status: character.status || statusFromAlive || statusFromMarvel || "Не
209     раскрыто",
210     alias: character.nameor || character.alias || "Не раскрыто",
211     universe: universeFromMarvel,
212     comicsCount:
213     Array.isArray(character.comics) ? character.comics.length :
214     character.comics && Array.isArray(character.comics.items) ? character.
215     comics.items.length :
216     0,
217     seriesCount:
218     Array.isArray(character.series) ? character.series.length :
219     character.series && Array.isArray(character.series.items) ? character.
220     series.items.length :
221     0,
222     modified: character.modified || "Не указано",
223     image: character.image || character.photo || imageFromThumbnail ||
224     FALLBACK_IMAGE,
225     description:
226     character.description ||

```

```

222     character.biography ||
223     "Описание отсутствует"
224 };
225 }
226
227 async function fetchCharacters() {
228     let lastError = null;
229
230     for (const url of API_URLS) {
231         try {
232             const response = await fetch(url);
233             if (!response.ok) throw new Error("HTTP " + response.status);
234
235             const payload = await response.json();
236             const rawCharacters = Array.isArray(payload)
237                 ? payload
238                 : payload.characters || [];
239             return rawCharacters.map(normalizeCharacter);
240         } catch (error) {
241             lastError = error;
242         }
243     }
244
245     throw lastError || new Error("Не удалось получить данные с API");
246 }
247
248 function escapeHtml(value) {
249     return String(value)
250         .replaceAll("&", "&amp;")
251         .replaceAll("<", "&lt;")
252         .replaceAll(">", "&gt;")
253         .replaceAll("'", "&quot;")
254         .replaceAll('"', "&#39;");
255 }
256
257 function getCharacterCard(character) {
258     return (
259         '<article class="character-card">' +
260         '' +
262         '<div class="body">' +
263         '<h3>' + escapeHtml(character.name) + "</h3>" +
264         '<p class="meta"><bТип>:</b> ' + escapeHtml(character.type) + "</p>" +
265         '<p class="meta"><bСтатус>:</b> ' + escapeHtml(character.status) + "</p>" +

```

```

265     '<button class="btn btn-secondary" data-open-modal="' + escapeHtml (
266         character.id) + 'Подробнее"></button>' +
267     "</div>" +
268     "</article>"
269 );
270 }
271 function getCharacterCards(characters) {
272     return characters.map(getCharacterCard);
273 }
274
275 function getCharacterModal(character) {
276     return (
277         '<div class="modal" id="modal-' + escapeHtml(character.id) + '" data-modal>'
278         +
279         '<div class="modal-box">' +
280         '<div class="modal-head">' +
281         "<h3>" + escapeHtml(character.name) + "</h3>" +
282         '<button class="close" data-close-modal>×</button>' +
283         "</div>" +
284         '<div class="modal-content">' +
285         '' +
287         "<p><bТип>:</b> " + escapeHtml(character.type) + "</p>" +
288         "<p><bСтатус>:</b> " + escapeHtml(character.status) + "</p>" +
289         "<p><bПсевдоним>:</b> " + escapeHtml(character.alias) + "</p>" +
290         "<p><bВселенная>:</b> " + escapeHtml(character.universe) + "</p>" +
291         "<p><bКомиксы>:</b> " + escapeHtml(character.comicsCount) + "</p>" +
292         "<p><bСерии>:</b> " + escapeHtml(character.seriesCount) + "</p>" +
293         "<p><bОбновлено>:</b> " + escapeHtml(character.modified) + "</p>" +
294         "<p><bОписание>:</b> " + escapeHtml(character.description) + "</p>" +
295         "</div>" +
296         "</div>" +
297         "</div>"
298     );
299 }
300
301 function getCharacterModals(characters) {
302     return characters.map(getCharacterModal);
303 }
304
305 function closeAllModals() {
306     document.querySelectorAll("[data-modal]").forEach((modal) => {
307         modal.classList.remove("is-open");
308     });
309 }

```

```

306     });
307 }
308
309 function bindModalEvents() {
310     cardsEl.addEventListener("click", (event) => {
311         const button = event.target.closest("[data-open-modal]");
312         if (!button) return;
313         const id = button.getAttribute("data-open-modal");
314         const modal = document.getElementById("modal-" + id);
315         if (modal) modal.classList.add("is-open");
316     });
317
318     modalsRoot.addEventListener("click", (event) => {
319         if (event.target.closest("[data-close-modal]")) {
320             closeAllModals();
321             return;
322         }
323         if (event.target.matches("[data-modal]")) {
324             closeAllModals();
325         }
326     });
327
328     document.addEventListener("keydown", (event) => {
329         if (event.key === "Escape") closeAllModals();
330     });
331 }
332
333 async function initLesson3() {
334     bindModalEvents();
335     try {
336         const characters = await fetchCharacters();
337         cardsEl.innerHTML = getCharacterCards(characters).join("");
338         modalsRoot.innerHTML = getCharacterModals(characters).join("");
339         statusEl.textContent = "Загружено персонажей: " + characters.length;
340     } catch (error) {
341         cardsEl.innerHTML = getCharacterCards(DEMO_CHARACTERS).join("");
342         modalsRoot.innerHTML = getCharacterModals(DEMO_CHARACTERS).join("");
343         statusEl.textContent =
344             "API недоступно, включен деморежим -: " +
345             (error && error.message ? error.message : error);
346     }
347 }
348
349 runLesson1();

```

```
350 runLesson2();
351 initLesson3();
352 runLesson4();
```

Листинг 1.3 — Файл `public/js/app.js`.

1.5.4 Стили интерфейса

```
1 * {
2   box-sizing: border-box;
3 }
4
5 body {
6   margin: 0;
7   background: #0f1220;
8   color: #f1f5f9;
9   font-family: "Segoe UI", Arial, sans-serif;
10 }
11
12 .header {
13   padding: 24px 16px;
14   text-align: center;
15   background: #151a2e;
16   border-bottom: 1px solid #293252;
17 }
18
19 .header h1 {
20   margin: 0 0 8px;
21   font-size: 28px;
22 }
23
24 .header p {
25   margin: 0;
26   color: #cbd5e1;
27 }
28
29 .container {
30   max-width: 1200px;
31   margin: 0 auto;
32   padding: 20px 16px 36px;
33   display: grid;
34   gap: 16px;
35 }
```

```
36
37 .card {
38   background: #151a2e;
39   border: 1px solid #293252;
40   border-radius: 12px;
41   padding: 16px;
42 }
43
44 .card h2 {
45   margin-top: 0;
46 }
47
48 .btn {
49   border: none;
50   border-radius: 8px;
51   background: #2563eb;
52   color: #fff;
53   padding: 10px 14px;
54   cursor: pointer;
55   font-weight: 600;
56 }
57
58 .btn:hover {
59   background: #1e4fb8;
60 }
61
62 .console {
63   margin-top: 12px;
64   min-height: 180px;
65   background: #090c17;
66   border: 1px solid #263049;
67   border-radius: 8px;
68   padding: 12px;
69   white-space: pre-wrap;
70   line-height: 1.45;
71   color: #dbeafe;
72 }
73
74 .status {
75   margin-top: 0;
76   color: #93c5fd;
77 }
78
79 .deploy-list {
```

```

80     margin: 12px 0 0;
81     padding-left: 22px;
82     color: #dbeafe;
83     line-height: 1.55;
84 }
85
86 .deploy-list li + li {
87     margin-top: 8px;
88 }
89
90 .cards {
91     display: grid;
92     grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
93     gap: 14px;
94 }
95
96 .character-card {
97     background: #0c1120;
98     border: 1px solid #2a3452;
99     border-radius: 10px;
100    overflow: hidden;
101 }
102
103 .character-card img {
104     display: block;
105     width: 100%;
106     height: 280px;
107     object-fit: cover;
108     background: #1f2937;
109 }
110
111 .character-card .body {
112     padding: 10px;
113 }
114
115 .character-card h3 {
116     margin: 0 0 8px;
117     font-size: 18px;
118 }
119
120 .meta {
121     margin: 0 0 4px;
122     color: #d1d5db;
123     font-size: 14px;

```

```
124 }
125
126 .btn-secondary {
127     margin-top: 8px;
128     width: 100%;
129     background: #0ea5e9;
130 }
131
132 .btn-secondary:hover {
133     background: #0284c7;
134 }
135
136 .modal {
137     position: fixed;
138     inset: 0;
139     display: none;
140     align-items: center;
141     justify-content: center;
142     padding: 16px;
143     background: rgba(0, 0, 0, 0.65);
144     z-index: 1000;
145 }
146
147 .modal.is-open {
148     display: flex;
149 }
150
151 .modal-box {
152     width: min(760px, 100%);
153     max-height: 90vh;
154     overflow: auto;
155     border-radius: 12px;
156     background: #151a2e;
157     border: 1px solid #2a3452;
158 }
159
160 .modal-head {
161     display: flex;
162     justify-content: space-between;
163     align-items: center;
164     padding: 12px 14px;
165     border-bottom: 1px solid #2a3452;
166 }
167
```

```
168 .close {
169     border: none;
170     background: transparent;
171     color: #e2e8f0;
172     font-size: 24px;
173     cursor: pointer;
174 }
175
176 .modal-content {
177     padding: 14px;
178 }
179
180 .modal-content img {
181     width: 220px;
182     max-width: 100%;
183     border-radius: 10px;
184     margin-bottom: 10px;
185 }
```

Листинг 1.4 — Файл `public/css/styles.css`.

1.6 Выводы

В ходе лабораторной работы №13 были реализованы базовые алгоритмические задачи на JavaScript, динамический пользовательский интерфейс и взаимодействие с внешним API. Итоговый проект приведён к структуре, пригодной для запуска и дальнейшей публикации на хостинге.

ЗАКЛЮЧЕНИЕ

В лабораторной работе №13 последовательно выполнены задания по базовым конструкциям JavaScript, работе со строками и массивами, обработке данных и построению интерактивного интерфейса. Дополнительно проект оформлен как готовое к запуску Node.js-приложение с Express-сервером и структурой, удобной для дальнейшей публикации на хостинге.